AgendaVet — Guia Completo do Fundador

Tudo que você precisa saber sobre o projeto que está construindo

Para quem é este guia? Para você, Wesley — o fundador do AgendaVet. Você não precisa saber programar para entender o que foi construído, como funciona, e como trabalhar de forma organizada para levar o produto até o lançamento. Este guia é baseado 100% no seu projeto real, com exemplos das pastas e arquivos reais do AgendaVet.

Índice

1. O que é o AgendaVet — visão geral
2. As 3 aplicações e para quem servem
3. O banco de dados — onde tudo é guardado
4. A inteligência artificial no projeto
5. Git — o que é, por que existe, como usar
6. Branches — o conceito que mais confunde
7. Passo a passo: do código ao ar (deploy)
8. Como trabalhar com Claude de forma organizada

9. Mapa completo do que já foi construído
10. Glossário — termos técnicos em português simples

---

1. O que é o AgendaVet

O AgendaVet é um **sistema de gestão para clínicas veterinárias**. Pense nele como um sistema que resolve três problemas ao mesmo tempo:

PROBLEMA	SOLUÇÃO NO AGENDAVET
Veterinário precisa registrar consultas, vacinas, cirurgias	App do Veterinário (AgendaVetVet)
Dono do pet quer ver o histórico e agendar consultas	App do Tutor (AgendaVetTutor)
Clínica precisa gerenciar agenda, financeiro e equipe	Painel Web (AgendaVetWeb)

Tudo isso conectado a **um único banco de dados** na nuvem (Supabase), então uma informação registrada no app do veterinário aparece automaticamente no painel web e no app do tutor.

A tecnologia em linguagem simples

- **Next.js** — o motor do painel web (como o motor de um carro)
- **Expo / React Native** — o motor dos apps mobile (iOS e Android)
- **Supabase** — o banco de dados na nuvem (onde tudo fica guardado)
- **Vercel** — o serviço que coloca o painel web no ar automaticamente
- **TypeScript** — a linguagem usada para escrever o código

---

2. As 3 Aplicações

2.1 AgendaVetWeb — O Painel da Clínica

Pasta: `AgendaVetWeb/`

Este é o "escritório digital" da clínica. Funciona no navegador (Chrome, Safari, etc.) e é onde o recepcionista e o gestor da clínica trabalham.

O que já tem pronto:

```
Tela de Login          → AgendaVetWeb/app/(auth)/login/page.tsx
Dashboard Principal    → AgendaVetWeb/app/page.tsx
Lista de Pets          → AgendaVetWeb/app/pets/page.tsx
Perfil do Pet          → AgendaVetWeb/app/pets/[id]/page.tsx
Prontuário do Pet      → AgendaVetWeb/app/pets/[id]/prontuario/page.tsx
Agendamentos           → AgendaVetWeb/app/appointments/page.tsx
Calendário             → AgendaVetWeb/app/calendar/page.tsx
Tutores (Donos)        → AgendaVetWeb/app/owners/page.tsx
Prontuários            → AgendaVetWeb/app/medical-records/page.tsx
Produtos e Serviços    → AgendaVetWeb/app/products-services/page.tsx
Analytics              → AgendaVetWeb/app/analytics/page.tsx
Copiloto IA            → AgendaVetWeb/app/vet-copilot/page.tsx
Configurações          → AgendaVetWeb/app/settings/page.tsx
```

Os módulos de atendimento (os formulários de registro clínico):

```
Consulta               → components/admin/modules/consulta-dialog.tsx
Vacina                 → components/admin/modules/vacina-dialog.tsx
Exame                  → components/admin/modules/exame-dialog.tsx
Receita                → components/admin/modules/receita-dialog.tsx
Peso                   → components/admin/modules/peso-dialog.tsx
Cirurgia               → components/admin/modules/cirurgia-dialog.tsx
Internação             → components/admin/modules/internacao-dialog.tsx
Banho e Tosa           → components/admin/modules/banho-tosa-dialog.tsx
Óbito                  → components/admin/modules/obito-dialog.tsx
Galeria de Fotos       → components/admin/modules/galeria-dialog.tsx
```

2.2 AgendaVetVet — O App do Veterinário

Pasta: `AgendaVetVet/`

Este é o app que o veterinário usa no celular durante as consultas. Ele consegue registrar tudo na tela do celular, na frente do paciente.

Telas prontas:

```
Login                  → AgendaVetVet/app/login.tsx
Dashboard              → AgendaVetVet/app/(tabs)/index.tsx
Lista de Pacientes     → AgendaVetVet/app/(tabs)/pacientes.tsx
Prontuário             → AgendaVetVet/app/(tabs)/prontuario.tsx
Solicitações           → AgendaVetVet/app/(tabs)/solicitacoes.tsx
```

Módulos clínicos (telas de registro):

```
Consulta               → AgendaVetVet/app/pet/consulta.tsx
Vacina                 → AgendaVetVet/app/pet/vacina.tsx
Exame                  → AgendaVetVet/app/pet/exame.tsx
Receita                → AgendaVetVet/app/pet/receita.tsx
Peso                   → AgendaVetVet/app/pet/peso.tsx
Cirurgia               → AgendaVetVet/app/pet/cirurgia.tsx
Internação             → AgendaVetVet/app/pet/internacao.tsx
Diagnóstico            → AgendaVetVet/app/pet/diagnostico.tsx
Banho e Tosa           → AgendaVetVet/app/pet/banho_tosa.tsx
Óbito                  → AgendaVetVet/app/pet/obito.tsx
Pagamento              → AgendaVetVet/app/pet/pagamento.tsx
Copiloto IA            → AgendaVetVet/app/pet/vet-copilot.tsx
```

2.3 AgendaVetTutor — O App do Dono do Pet

Pasta: `AgendaVetTutor/`

Este é o app do cliente da clínica — o dono do animal. Ele pode ver o histórico do pet, agendar consultas e receber documentos.

Telas prontas:

```
Login                  → AgendaVetTutor/app/login.tsx
Home                   → AgendaVetTutor/app/(tabs)/index.tsx
Agendamentos           → AgendaVetTutor/app/(tabs)/agendamentos.tsx
Perfil                 → AgendaVetTutor/app/(tabs)/perfil.tsx
Financeiro             → AgendaVetTutor/app/(tabs)/financeiro.tsx
```

```
Detalhes do Pet       → AgendaVetTutor/app/pet-details/[id].tsx
Assinar Documento     → AgendaVetTutor/app/pet-details/assinar-documento.tsx
```

2.4 Os Pacotes Compartilhados

Além dos 3 apps, existe uma "caixa de ferramentas compartilhada":

```
shared/          → Tipos e funções usadas pelos 3 apps ao mesmo tempo
packages/theme/  → As cores e estilos visuais compartilhados
```

Isso evita que o mesmo código seja escrito 3 vezes. Se muda em `shared/`, muda automaticamente nos 3 apps.

---

3. O Banco de Dados

O que é o Supabase

O Supabase é o "cofre" do AgendaVet. Fica na nuvem (na internet) e guarda todos os dados: pets, tutores, consultas, vacinas, tudo.

Funciona com **tabelas**, como planilhas do Excel, onde cada linha é um registro e cada coluna é uma informação.

As 21 tabelas do AgendaVet

TABELA	O QUE GUARDA
`profiles`	Usuários do sistema (tutores e admins)

`user_roles`	Quem é admin, quem é tutor
`pets`	Dados dos animais
`services`	Catálogo de serviços da clínica
`appointment_requests`	Agendamentos e atendimentos
`anamnesis`	Fichas de consulta clínica
`pet_admin_history`	Histórico de todas as ações
`pet_weight_records`	Histórico de peso dos pets
`pet_pathologies`	Doenças e condições
`pet_documents`	Documentos anexados
`pet_exams`	Resultados de exames
`pet_photos`	Fotos dos pets
`pet_vaccines`	Registro de vacinas
`pet_prescriptions`	Receitas médicas
`pet_observations`	Observações clínicas
`pet_videos`	Vídeos gravados
`pet_hospitalizations`	Internações
`mortes`	Registros de óbito
`audit_logs`	Trilha de auditoria
`pet_services`	Serviços prestados

Segurança: o RLS

Cada tabela tem um sistema de segurança chamado **RLS (Row Level Security)**. Em termos simples:

- **Admin** (funcionário da clínica) → vê e altera tudo
- **Tutor** (dono do pet) → vê apenas os dados dos seus próprios pets

Isso garante que um tutor nunca consiga ver os dados do pet de outra pessoa.

---

4. A Inteligência Artificial

O AgendaVet tem um sistema de IA chamado **Vet Copilot** — um assistente clínico inteligente integrado ao app do veterinário e ao painel web.

Os modelos de IA usados

MODELO	EMPRESA	PARA QUE SERVE NO AGENDAVET
DeepSeek R1	DeepSeek	Raciocínio clínico, diagnóstico diferencial
Gemini 2.0/2.5 Flash	Google	Respostas rápidas, triagem de emergência
Claude Sonnet	Anthropic	Análise de imagens (radiografias, fotos)
GPT-4o	OpenAI	Planejamento cirúrgico

O que o Copiloto faz

- Sugere diagnósticos diferenciais com base nos sintomas
- Analisa imagens médicas
- Auxilia no planejamento de cirurgias
- Lembra do histórico da conversa
- Controla o custo de tokens automaticamente
- Tem permissões por tipo de usuário

Os arquivos do Copiloto ficam em:

```
AgendaVetWeb/lib/vet-copilot/
  ai-gateway.ts         → Decide qual IA usar em cada situação
  system-prompt.ts      → As instruções que guiam o comportamento da IA
  rag.ts                → Busca em documentos para enriquecer respostas
  cost-controller.ts    → Controla o gasto com APIs de IA
  memory-manager.ts     → Guarda o histórico da conversa
  tools.ts              → O que a IA pode "fazer" (ler, buscar, calcular)
```

---

5. Git Sem Complicação

O que é o Git

Git é um sistema de **controle de versões**. Pense nele como um histórico de salvamentos de um documento, mas muito mais poderoso.

Toda vez que uma mudança é salva no Git, isso cria uma **foto** do projeto naquele momento. Você pode voltar a qualquer foto anterior se algo der errado.

O histórico real do AgendaVet

Cada linha abaixo é um "salvamento" (chamado de **commit**) do seu projeto:

```
0d70235  fix: reduz tamanho dos botões de salvar nos dialogs
9b60d04  fix: redimensiona botões gigantes nos módulos de atendimento
c5e7149  fix(calendar): encurta painel flutuante
4e6b0be  feat(calendar): transforma faixa de detalhes em painel flutuante
918a622  fix: corrige botão 'Adicionar Tutor' que não salvava no banco
eebb1b7  feat(products-services): split Serviços/Produtos com tabs
...
```

Cada linha tem:

- Um **código** (ex: `0d70235`) — o "número de série" do salvamento
- Uma **mensagem** — o que foi mudado

Os 3 comandos mais importantes

```
git status       # Mostra o que foi modificado mas ainda não foi salvo
git add .        # Prepara tudo para ser salvo
git commit -m "mensagem"   # Salva com uma descrição
git push         # Envia para o GitHub (a nuvem)
```

Você não precisa rodar esses comandos — o Claude faz isso. Mas entender o que cada um faz ajuda a compreender o fluxo.

---

6. Branches Sem Medo

O que é uma branch

Imagine que o seu projeto é uma árvore. O tronco principal é a `master` (ou `main`). Uma **branch** é um galho — uma cópia separada do tronco onde você pode trabalhar sem afetar o tronco.

```
master ──────────────────────────────── (produção, o que está no ar)
          │
          └── claude/check-agendavetweb-folder-JBpJI  (onde o Claude trabalha agora)
```

Por que usar branches

- Para o Claude trabalhar em mudanças sem quebrar o que já está funcionando
- Para revisar e aprovar mudanças antes de colocar no ar
- Para múltiplas melhorias acontecerem em paralelo

As branches do seu projeto agora

```
master                                  → Branch principal (produção)
origin/main                             → Cópia na nuvem (GitHub)
claude/check-agendavetweb-folder-JBpJI  → Branch atual do Claude
```

Como as mudanças chegam ao master: o Pull Request

Quando o Claude termina uma tarefa numa branch, o caminho para o código chegar ao projeto oficial é:

```
1. Claude faz as mudanças na branch   claude/check-agendavetweb-folder-JBpJI
2. Claude faz push para o GitHub
3. Você abre um Pull Request no GitHub
4. Você revisa e clica em "Merge"
5. As mudanças entram no master
6. O Vercel detecta automaticamente e publica no ar
```

Passo a passo para fazer o merge no GitHub

```
1. Acesse: github.com/melowesley/AgendaVet

2. Clique em "Pull requests" no menu superior

3. Clique em "New pull request"

4. Configure assim:
   base: master
   compare: claude/check-agendavetweb-folder-JBpJI

5. Clique em "Create pull request"

6. Revise as mudanças listadas

7. Clique em "Merge pull request"

8. Clique em "Confirm merge"

Pronto. As mudanças estão no master.
```

O problema dos dois Claudes

Você mencionou que às vezes tem dois chats com Claude editando partes diferentes do projeto ao mesmo tempo.

Se estiverem em branches diferentes: sem problema algum.

Se estiverem na mesma branch: quando o segundo tentar fazer push, vai receber um erro. A solução é:

```
git pull origin [nome-da-branch]
# Isso "puxa" as mudanças do outro Claude
# Depois o push vai funcionar
```

A recomendação é: **cada sessão de Claude = uma branch diferente**. O padrão `claude/nome-da-tarefa-XXXXX` já garante isso automaticamente.

---

7. Do Código ao Ar

Como o AgendaVetWeb vai para a internet

O fluxo completo é:

```
Você pede mudança ao Claude
       ↓
Claude edita os arquivos na sua máquina
       ↓
Claude faz commit + push para o GitHub
       ↓
Você abre Pull Request e faz merge no master
       ↓
Vercel detecta o merge no master automaticamente
       ↓
Vercel faz o build e publica no ar (em ~2 minutos)
```

```
       ↓
O painel web está atualizado
```

Como os apps mobile vão para as lojas

Os apps mobile (Vet e Tutor) têm um processo diferente:

```
Código finalizado no master
       ↓
EAS Build (Expo Application Services)
       ↓
Gera o arquivo .apk (Android) ou .ipa (iOS)
       ↓
Envio para Google Play / App Store
       ↓
Aprovação da loja (pode levar horas ou dias)
       ↓
Disponível para download
```

Para publicar uma atualização de conteúdo nos apps sem passar pela loja, o Expo tem o **EAS Update** — que atualiza instantaneamente.

---

8. Trabalhando com Claude

O problema que você descreveu

Você disse que se perde com as branches, quebra fluxos e desestrutura o projeto. Isso acontece por um motivo simples: **você não tem um processo definido**.

Com um processo claro, tudo fica mais fácil.

O processo recomendado: uma tarefa por vez

```
REGRA 1: Uma conversa = Uma tarefa
  Não misture "arrumar o botão" com "criar nova tela" na mesma conversa.
  Finalize uma antes de começar a outra.

REGRA 2: Sempre faça merge antes de pedir uma nova tarefa grande
  Depois que o Claude termina e faz o push, vá ao GitHub,
  abra o PR e faça o merge. Só então peça a próxima tarefa.

REGRA 3: Descreva o resultado que quer, não como fazer
  ❌ "Preciso que você altere o className do Button para h-10"
  ✓  "Os botões de salvar estão muito grandes em todos os módulos"

REGRA 4: Um Claude por branch
  Se você abrir dois chats simultaneamente, certifique-se de que
  estão editando pastas completamente diferentes.
```

Como pedir tarefas de forma eficiente

Modelo de pedido ideal:

```
Contexto: [onde está o problema]
Problema: [o que está errado ou faltando]
Resultado esperado: [como deve ficar depois]

Exemplo real:
"Contexto: No módulo de Consulta do painel web
 Problema: O botão 'Salvar Registro' está muito grande (h-14)
 Resultado: Deve ter o mesmo tamanho dos botões normais do sistema (h-10)"
```

Como verificar o que o Claude fez

Antes de fazer o merge de um Pull Request, você pode ver exatamente o que mudou:

```
No GitHub → aba "Files changed" do Pull Request
Linhas em vermelho = o que foi removido
Linhas em verde    = o que foi adicionado
```

9. O que Já Está Pronto

Resumo executivo

O AgendaVet está em estágio avançado de desenvolvimento. A estrutura completa foi construída e a maioria das funcionalidades principais está implementada.

Funcionalidades prontas

Painel Web (AgendaVetWeb):

- ☑ Autenticação (login/logout)
- ☑ Dashboard com visão geral
- ☑ Gestão de pets (criar, editar, ver histórico)
- ☑ Gestão de tutores (donos dos pets)
- ☑ Calendário de agendamentos
- ☑ 12 módulos clínicos (consulta, vacina, exame, receita, peso, cirurgia, internação, banho, óbito, galeria, exame, arquivo)
- ☑ Prontuário unificado (timeline de todos os registros)
- ☑ Analytics (taxas de ocupação, receita, picos de horário)
- ☑ Produtos e Serviços com abas separadas
- ☑ Copiloto de IA com múltiplos modelos
- ☑ Sistema de impressão/PDF dos registros

App do Veterinário (AgendaVetVet):

- ☑ Autenticação
- ☑ Lista de pacientes com filtros
- ☑ Todos os módulos clínicos em mobile
- ☑ Geração de PDF no celular
- ☑ Copiloto de IA
- ☑ Calendário
- ☑ Analytics mobile
- ☑ Deep linking (navegação entre telas via link)

App do Tutor (AgendaVetTutor):

- ☑ Autenticação
- ☑ Home com resumo dos pets
- ☑ Visualização do histórico do pet
- ☑ Agendamentos
- ☑ Financeiro
- ☑ Assinatura digital de documentos
- ☑ Visualização de documentos

Banco de Dados:

- ☑ 21 tabelas com schema completo
- ☑ Segurança RLS configurada
- ☑ 12 migrações versionadas
- ☑ Políticas de acesso por perfil

Inteligência Artificial:

- ☑ 4 modelos de IA integrados
- ☑ Sistema RAG (busca em documentos)
- ☑ Memória de conversa
- ☑ Controle de custo automático
- ☑ Prompts especializados (diagnóstico, sintomas, tratamento)

O que pode faltar para o lançamento

Isso você precisa avaliar com base no seu produto. Perguntas para se guiar:

1. **UX/UI:** O visual está satisfatório em todas as telas?
2. **Testes reais:** Você já usou com um veterinário real em uma clínica real?
3. **Onboarding:** Como uma nova clínica cria sua conta e configura o sistema?
4. **Suporte:** Existe algum canal para clientes reportarem problemas?
5. **Deploy dos apps:** Os apps estão nas lojas (Google Play / App Store)?
6. **Domínio:** O painel web tem um domínio próprio (ex: app.agendavet.com.br)?

---

10. Glossário

TERMO	SIGNIFICADO SIMPLES
Branch	Um galho do projeto — cópia isolada para trabalhar sem afetar o principal
Commit	Um "salvamento" com descrição do que foi mudado
Push	Enviar os commits da máquina local para o GitHub (a nuvem)
Pull Request (PR)	Um pedido para incorporar mudanças de uma branch para outra
Merge	A fusão — quando as mudanças de uma branch entram na outra
Master / Main	O tronco principal do projeto (o que vai para produção)
Deploy	O processo de colocar o código no ar (na internet)
Build	O processo de "compilar" o código para poder publicá-lo
Vercel	Serviço que hospeda o painel web e faz deploy automático
Supabase	O banco de dados na nuvem do AgendaVet
RLS	Row Level Security — segurança que define quem vê o quê no banco
API	Uma "janela" que permite que dois sistemas se comuniquem
TypeScript	A linguagem de programação usada no AgendaVet
Next.js	O framework do painel web
Expo	O framework dos apps mobile
Monorepo	Um único repositório Git que contém múltiplos projetos
Hook	Código que executa automaticamente quando algo acontece
Middleware	Código que roda "no meio" — entre a requisição e a resposta
Component	Um bloco reutilizável de interface (botão, formulário, card)
Token	Unidade de custo das APIs de IA (aproximadamente 1 palavra = 1 token)

RAG	Técnica de IA que busca em documentos para enriquecer respostas
Lint	Ferramenta que verifica erros de código automaticamente
Zustand	Biblioteca de gerenciamento de estado do painel web
Tailwind CSS	Sistema de estilização visual usado nos 3 apps
EAS	Expo Application Services — serviço para publicar apps nas lojas

Apêndice: Estrutura de Pastas Completa

```
/AgendaVet/
│
├── AgendaVetWeb/                    # Painel Web (Next.js)
│   ├── app/                         # Páginas e rotas
│   │   ├── (auth)/login/            # Tela de login
│   │   ├── pets/                    # Gestão de pets
│   │   ├── appointments/            # Agendamentos
│   │   ├── calendar/                # Calendário
│   │   ├── owners/                  # Tutores
│   │   ├── analytics/               # Relatórios
│   │   ├── products-services/       # Serviços e produtos
│   │   ├── vet-copilot/             # IA Copiloto
│   │   ├── settings/                # Configurações
│   │   └── api/                     # Endpoints de API
│   ├── components/                  # Componentes visuais
│   │   ├── admin/modules/           # Módulos clínicos (12 formulários)
│   │   ├── ui/                      # Componentes de interface base
│   │   └── ...
│   ├── lib/                         # Funções e lógica
│   │   ├── supabase/                # Conexão com banco de dados
│   │   ├── vet-copilot/             # Motor da IA
│   │   └── ...
│   └── vercel.json                  # Config de deploy
│
├── AgendaVetVet/                    # App Veterinário (Expo)
│   ├── app/                         # Telas
│   │   ├── (tabs)/                  # Tabs principais
│   │   ├── pet/                     # Módulos clínicos mobile
│   │   └── menu/                    # Telas do menu
│   ├── lib/                         # Lógica e serviços
│   │   ├── vet-copilot/             # IA no mobile
│   │   ├── pdf/                     # Geração de PDF
```

```
│   │   └── ...
│   └── app.json                    # Config do app
│
├── AgendaVetTutor/                 # App do Dono do Pet (Expo)
│   ├── app/                        # Telas
│   │   ├── (tabs)/                 # Tabs principais
│   │   └── pet-details/            # Detalhes e documentos
│   └── app.json                    # Config do app
│
├── shared/                         # Código compartilhado
│   └── exports/
│       ├── types.ts                # Tipos de dados
│       ├── constants.ts            # Constantes do sistema
│       └── utils.ts                # Funções utilitárias
│
├── packages/theme/                 # Tema visual compartilhado
│
└── supabase/                       # Banco de dados
    ├── migrations/                 # 12 versões do schema
    └── functions/                  # Funções serverless
```

Como converter este arquivo em PDF

Este guia está no formato Markdown (`.md`). Para transformar em PDF:

Opção 1 — VSCode (recomendado):

1. Abra o arquivo `GUIA-AGENDAVET.md` no VSCode
2. Pressione `Ctrl+Shift+P` (ou `Cmd+Shift+P` no Mac)
3. Digite "Markdown: Open Preview"
4. Na aba de preview, clique com botão direito → "Print" → "Save as PDF"

Opção 2 — Navegador:

1. Acesse: https://markdownlivepreview.com
2. Cole o conteúdo deste arquivo
3. Use `Ctrl+P` → "Salvar como PDF"

Opção 3 — GitHub: O GitHub renderiza arquivos `.md` automaticamente com formatação. Acesse: github.com/melowesley/AgendaVet/blob/master/GUIA-AGENDAVET.md

---

Guia gerado em 17 de março de 2026 com base no estado atual do projeto AgendaVet. Atualizado por Claude Code — Anthropic.